In [1]:
```
import pandas as pd
```

In [2]:
```
movie =pd.read_csv(r'C:\Users\pandi\Desktop\Abhiraj\Data Science\Notes\Learning\1st
```

In [3]:
```
movie
```

Out[3]:

	movieId	title	genres
0	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
1	2	Jumanji (1995)	Adventure|Children|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	4	Waiting to Exhale (1995)	Comedy|Drama|Romance
4	5	Father of the Bride Part II (1995)	Comedy
...	...	...	...
27273	131254	Kein Bund für's Leben (2007)	Comedy
27274	131256	Feuer, Eis & Dosenbier (2002)	Comedy
27275	131258	The Pirates (2014)	Adventure
27276	131260	Rentun Ruusu (2001)	(no genres listed)
27277	131262	Innocence (2014)	Adventure|Fantasy|Horror

27278 rows × 3 columns

In [4]:
```
tag =pd.read_csv(r'C:\Users\pandi\Desktop\Abhiraj\Data Science\Notes\Learning\1st,
```

In [5]:
```
tag
```

Out[5]:

	userId	movieId	tag	timestamp
0	18	4141	Mark Waters	2009-04-24 18:19:40
1	65	208	dark hero	2013-05-10 01:41:18
2	65	353	dark hero	2013-05-10 01:41:19
3	65	521	noir thriller	2013-05-10 01:39:43
4	65	592	dark hero	2013-05-10 01:41:18
...	...	...	...	...
465559	138446	55999	dragged	2013-01-23 23:29:32
465560	138446	55999	Jason Bateman	2013-01-23 23:29:38
465561	138446	55999	quirky	2013-01-23 23:29:38
465562	138446	55999	sad	2013-01-23 23:29:32
465563	138472	923	rise to power	2007-11-02 21:12:47

465564 rows × 4 columns

In [6]:
```
rating = pd.read_csv(r'C:\Users\pandi\Desktop\Abhiraj\Data Science\Notes\Learning\1
```

In [7]:
```
rating.head()
```

Out[7]:

	userId	movieId	rating	timestamp
0	1	2	3.5	2005-04-02 23:53:47
1	1	29	3.5	2005-04-02 23:31:16
2	1	32	3.5	2005-04-02 23:33:39
3	1	47	3.5	2005-04-02 23:32:07
4	1	50	3.5	2005-04-02 23:29:40

In [8]:
```
del rating['timestamp']
del tag['timestamp']
```

In [9]:
```
tag_0 = tag.iloc[0]
```

In [10]:
```
tag.iloc[0]
```

Out[10]:
```
userId              18
movieId           4141
tag        Mark Waters
Name: 0, dtype: object
```

In [11]:
```
type(tag_0)
```

Out[11]: pandas.core.series.Series

In [12]:
```
tag_0.index
```

Out[12]: Index(['userId', 'movieId', 'tag'], dtype='object')

In [13]:
```
tag_0['userId']
```

Out[13]: np.int64(18)

In [14]:
```
'userId' in tag_0
```

Out[14]: True

In [15]:
```
tag_0.name
```

Out[15]: 0

In [16]:
```
tag_0.userId
```

Out[16]: np.int64(18)

In [17]:
```
tag_0['userId']
```

Out[17]: np.int64(18)

In [18]:
```
tag_0 = tag_0.rename('first_name')
```

In [19]:
```
tag_0.name
```

Out[19]: 'first_name'

In [20]:
```
tag.head()
```

Out[20]:

	userId	movieId	tag
0	18	4141	Mark Waters
1	65	208	dark hero
2	65	353	dark hero
3	65	521	noir thriller
4	65	592	dark hero

In [21]:
```
tag.columns
```

Out[21]: Index(['userId', 'movieId', 'tag'], dtype='object')

In [22]:
```
tag.iloc[[0,1,2,3]]
```

Out[22]:

	userId	movieId	tag
0	18	4141	Mark Waters
1	65	208	dark hero
2	65	353	dark hero
3	65	521	noir thriller

In [23]:
```
tag.iloc[:4]
```

Out[23]:

	userId	movieId	tag
0	18	4141	Mark Waters
1	65	208	dark hero
2	65	353	dark hero
3	65	521	noir thriller

In [24]:
```
rating
```

Out[24]:

	userId	movieId	rating
0	1	2	3.5
1	1	29	3.5
2	1	32	3.5
3	1	47	3.5
4	1	50	3.5
...	...	...	...
20000258	138493	68954	4.5
20000259	138493	69526	4.5
20000260	138493	69644	3.0
20000261	138493	70286	5.0
20000262	138493	71619	2.5

20000263 rows × 3 columns

In [25]:
```
rating.describe()
```

Out[25]:

	userId	movieId	rating
count	2.000026e+07	2.000026e+07	2.000026e+07
mean	6.904587e+04	9.041567e+03	3.525529e+00
std	4.003863e+04	1.978948e+04	1.051989e+00
min	1.000000e+00	1.000000e+00	5.000000e-01
25%	3.439500e+04	9.020000e+02	3.000000e+00
50%	6.914100e+04	2.167000e+03	3.500000e+00
75%	1.036370e+05	4.770000e+03	4.000000e+00
max	1.384930e+05	1.312620e+05	5.000000e+00

In [26]:
```
rating['rating'].describe()
```

Out[26]:
```
count    2.000026e+07
mean     3.525529e+00
std      1.051989e+00
min      5.000000e-01
25%      3.000000e+00
50%      3.500000e+00
75%      4.000000e+00
max      5.000000e+00
Name: rating, dtype: float64
```

In [27]:
```
rating.mean()
```

Out[27]:
```
userId     69045.872583
movieId     9041.567330
rating         3.525529
dtype: float64
```

In [28]:
```
rating['rating'].max()
```

Out[28]:
```
5.0
```

In [29]:
```
rating['rating'].min()
```

Out[29]:
```
0.5
```

In [30]:
```
rating['rating'].std()
```

Out[30]:
```
1.051988919275684
```

In [31]:
```
rating['rating'].mode()
```

Out[31]:
```
0    4.0
Name: rating, dtype: float64
```

In [32]:
```
rating.corr()   #correlation b/w columns
```

Out[32]:

	userId	movieId	rating
userId	1.000000	-0.000850	0.001175
movieId	-0.000850	1.000000	0.002606
rating	0.001175	0.002606	1.000000

In [33]:
```
filter1 = rating.rating > 10
```

In [34]:
```
filter1.any()
```

Out[34]: np.False_

In [35]:
```
filter2 = rating.rating>0
```

In [36]:
```
filter2.all()
```

Out[36]: np.True_

In [37]:
```
movie.shape
```

Out[37]: (27278, 3)

In [38]:
```
movie.isnull().any().any()   #representing no null value
```

Out[38]: np.False_

In [39]:
```
tag.isnull().any().any()  #tag have some null value
```

Out[39]: np.True_

In [40]:
```
rating.isnull().any().any()
```

Out[40]: np.False_

In [41]:
```
tag.isnull().any()
```

Out[41]:
```
userId     False
movieId    False
tag         True
dtype: bool
```

In [42]:
```
#tag['tag'] = 'none'
```

In [43]:
```
tag['tag'].isnull().sum()
```

Out[43]: np.int64(16)

In [44]:
```
tag = tag.dropna()
```

In [45]:
```
tag.isnull().any().any()
```

Out[45]:
```
np.False_
```

In [46]:
```
tag.shape     # drop the na records
```

Out[46]:
```
(465548, 3)
```

In [47]:
```
rating
```

Out[47]:

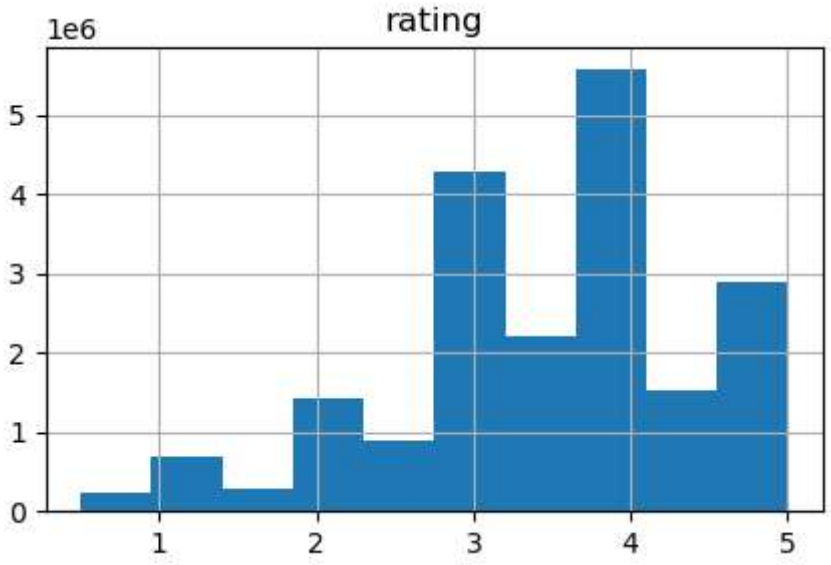
	userId	movieId	rating
0	1	2	3.5
1	1	29	3.5
2	1	32	3.5
3	1	47	3.5
4	1	50	3.5
...	...	...	...
20000258	138493	68954	4.5
20000259	138493	69526	4.5
20000260	138493	69644	3.0
20000261	138493	70286	5.0
20000262	138493	71619	2.5

20000263 rows × 3 columns

In [48]:
```
import matplotlib.pyplot as plt
%matplotlib inline
```

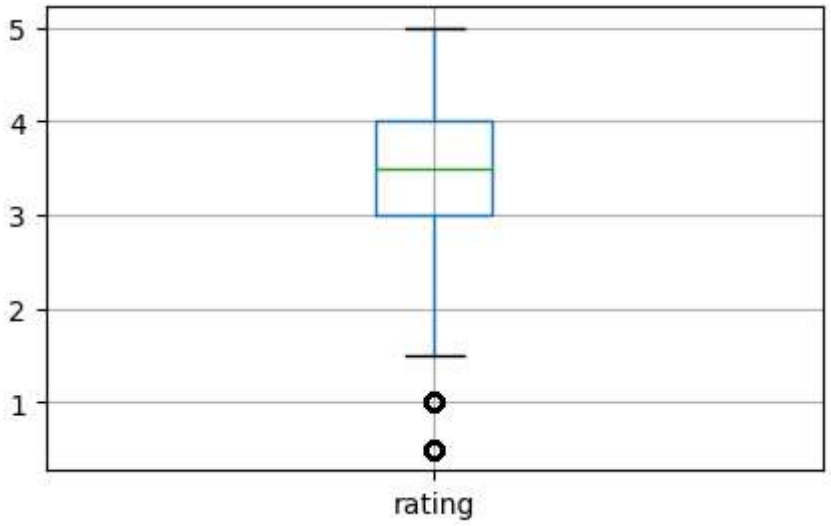
In [49]:
```
rating.hist(column = 'rating', figsize = (5,3))
```

Out[49]:
```
array([[<Axes: title={'center': 'rating'}>]], dtype=object)
```



In [50]:
```
rating.boxplot(column= 'rating', figsize = (5,3))
```

Out[50]: <Axes: >



In [51]:
```
movie.head()
```

Out[51]:

	movieId	title	genres
0	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
1	2	Jumanji (1995)	Adventure|Children|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	4	Waiting to Exhale (1995)	Comedy|Drama|Romance
4	5	Father of the Bride Part II (1995)	Comedy

In [52]:
```
movie[['title','genres']].head()
```

Out[52]:

	title	genres
0	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
1	Jumanji (1995)	Adventure|Children|Fantasy
2	Grumpier Old Men (1995)	Comedy|Romance
3	Waiting to Exhale (1995)	Comedy|Drama|Romance
4	Father of the Bride Part II (1995)	Comedy

In [53]:
```
movie[1:20]
```

Out[53]:

	movieId	title	genres
1	2	Jumanji (1995)	Adventure|Children|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	4	Waiting to Exhale (1995)	Comedy|Drama|Romance
4	5	Father of the Bride Part II (1995)	Comedy
5	6	Heat (1995)	Action|Crime|Thriller
6	7	Sabrina (1995)	Comedy|Romance
7	8	Tom and Huck (1995)	Adventure|Children
8	9	Sudden Death (1995)	Action
9	10	GoldenEye (1995)	Action|Adventure|Thriller
10	11	American President, The (1995)	Comedy|Drama|Romance
11	12	Dracula: Dead and Loving It (1995)	Comedy|Horror
12	13	Balto (1995)	Adventure|Animation|Children
13	14	Nixon (1995)	Drama
14	15	Cutthroat Island (1995)	Action|Adventure|Romance
15	16	Casino (1995)	Crime|Drama
16	17	Sense and Sensibility (1995)	Drama|Romance
17	18	Four Rooms (1995)	Comedy
18	19	Ace Ventura: When Nature Calls (1995)	Comedy
19	20	Money Train (1995)	Action|Comedy|Crime|Drama|Thriller

In [54]:
```
tag_count = tag['tag'].value_counts()
tag_count[-10:]
```

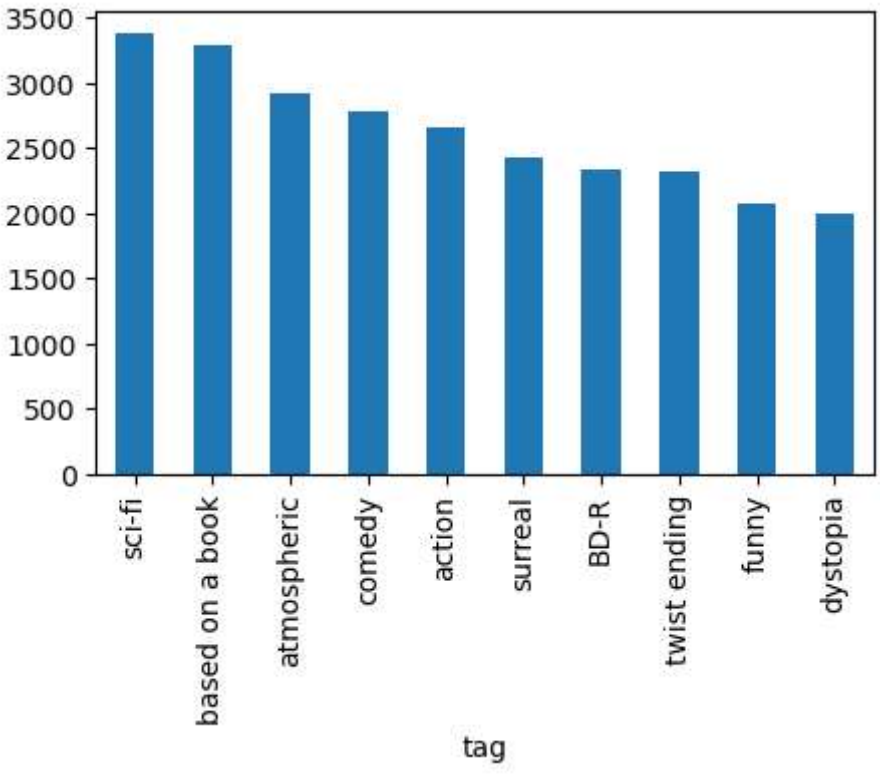
Out[54]:
```
tag
missing child                  1
Ron Moore                      1
Citizen Kane                   1
mullet                         1
biker gang                     1
Paul Adelstein                 1
the wig                        1
killer fish                    1
genetically modified monsters  1
topless scene                  1
Name: count, dtype: int64
```

In [55]:
```
tag['tag'].value_counts()
```

Out[55]:
```
tag
sci-fi                         3384
based on a book                3281
atmospheric                    2917
comedy                         2779
action                         2657
                               ...
Paul Adelstein                    1
the wig                           1
killer fish                       1
genetically modified monsters     1
topless scene                     1
Name: count, Length: 38643, dtype: int64
```

In [60]:
```
tag_count[:10].plot(kind = 'bar',figsize = (5,3))
```

Out[60]: <Axes: xlabel='tag'>



In [65]:
```
high_rating = rating.rating >= 5.0
```

In [68]:
```
rating[high_rating][10:20]
```

Out[68]:

	userId	movieId	rating
185	2	589	5.0
188	2	924	5.0
190	2	1196	5.0
191	2	1210	5.0
192	2	1214	5.0
193	2	1249	5.0
194	2	1259	5.0
195	2	1270	5.0
196	2	1327	5.0
197	2	1356	5.0

In [73]:
```
movie_flt = movie['genres'].str.contains('Action')
```

In [74]:
```
movie[movie_flt]
```

Out[74]:

	movieId	title	genres
5	6	Heat (1995)	Action|Crime|Thriller
8	9	Sudden Death (1995)	Action
9	10	GoldenEye (1995)	Action|Adventure|Thriller
14	15	Cutthroat Island (1995)	Action|Adventure|Romance
19	20	Money Train (1995)	Action|Comedy|Crime|Drama|Thriller
...	...	...	...
27168	130842	Power/Rangers (2015)	Action|Adventure|Sci-Fi
27187	130984	Santo vs. las lobas (1976)	Action|Fantasy|Horror
27198	131025	The Brass Legend (1956)	Action
27236	131122	Love Exposure (2007)	Action|Comedy|Drama|Romance
27264	131180	Dead Rising: Watchtower (2015)	Action|Horror|Thriller

3520 rows × 3 columns

In [82]:
```
rating[['movieId','rating']].groupby('rating').count()
```

Out[82]:

rating	movieId
0.5	239125
1.0	680732
1.5	279252
2.0	1430997
2.5	883398
3.0	4291193
3.5	2200156
4.0	5561926
4.5	1534824
5.0	2898660

In [85]:
```
rating[['movieId', 'rating']].groupby('rating').mean()
```

Out[85]:

rating	movieId
0.5	13356.246729
1.0	5652.208219
1.5	12377.773230
2.0	6733.595232
2.5	13669.268192
3.0	6770.763264
3.5	14814.098703
4.0	8342.514461
4.5	14585.414824
5.0	6275.356017

In [86]:
```
movie.head()
```

Out[86]:

	movieId	title	genres
0	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
1	2	Jumanji (1995)	Adventure|Children|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	4	Waiting to Exhale (1995)	Comedy|Drama|Romance
4	5	Father of the Bride Part II (1995)	Comedy

In [87]:
```
tag.head()
```

Out[87]:

	userId	movieId	tag
0	18	4141	Mark Waters
1	65	208	dark hero
2	65	353	dark hero
3	65	521	noir thriller
4	65	592	dark hero

In [88]:
```
t1 = movie.merge(tag, on ='movieId', how = 'inner')
```

In [90]:
```
t1.head()
```

Out[90]:

	movieId	title	genres	userId	tag
0	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy	1644	Watched
1	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy	1741	computer animation
2	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy	1741	Disney animated feature
3	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy	1741	Pixar animation
4	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy	1741	TÃ©a Leoni does not star in this movie

In [98]:
```
avg_rating = rating[['movieId','rating']].groupby('movieId').mean()
```

In [100…
```
avg_rating.head
```

Out[100…

```
<bound method NDFrame.head of            rating
movieId
1        3.921240
2        3.211977
3        3.151040
4        2.861393
5        3.064592
...           ...
131254   4.000000
131256   4.000000
131258   2.500000
131260   3.000000
131262   4.000000

[26744 rows x 1 columns]>
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: